

Android 13

Camera驱动配置指导手册

WWW.UNISOC.COM



修改历史



版本号	日期	注释
V1.0	2022/10/14	第一次正式发布。

关键字

关键字：Sensor驱动、AF驱动、OTP驱动、Flash驱动

目录



01 Camera驱动代码结构

02 Sensor驱动配置

03 OTP驱动配置

04 AF驱动配置

05 Flash驱动配置

06 驱动编译和下载

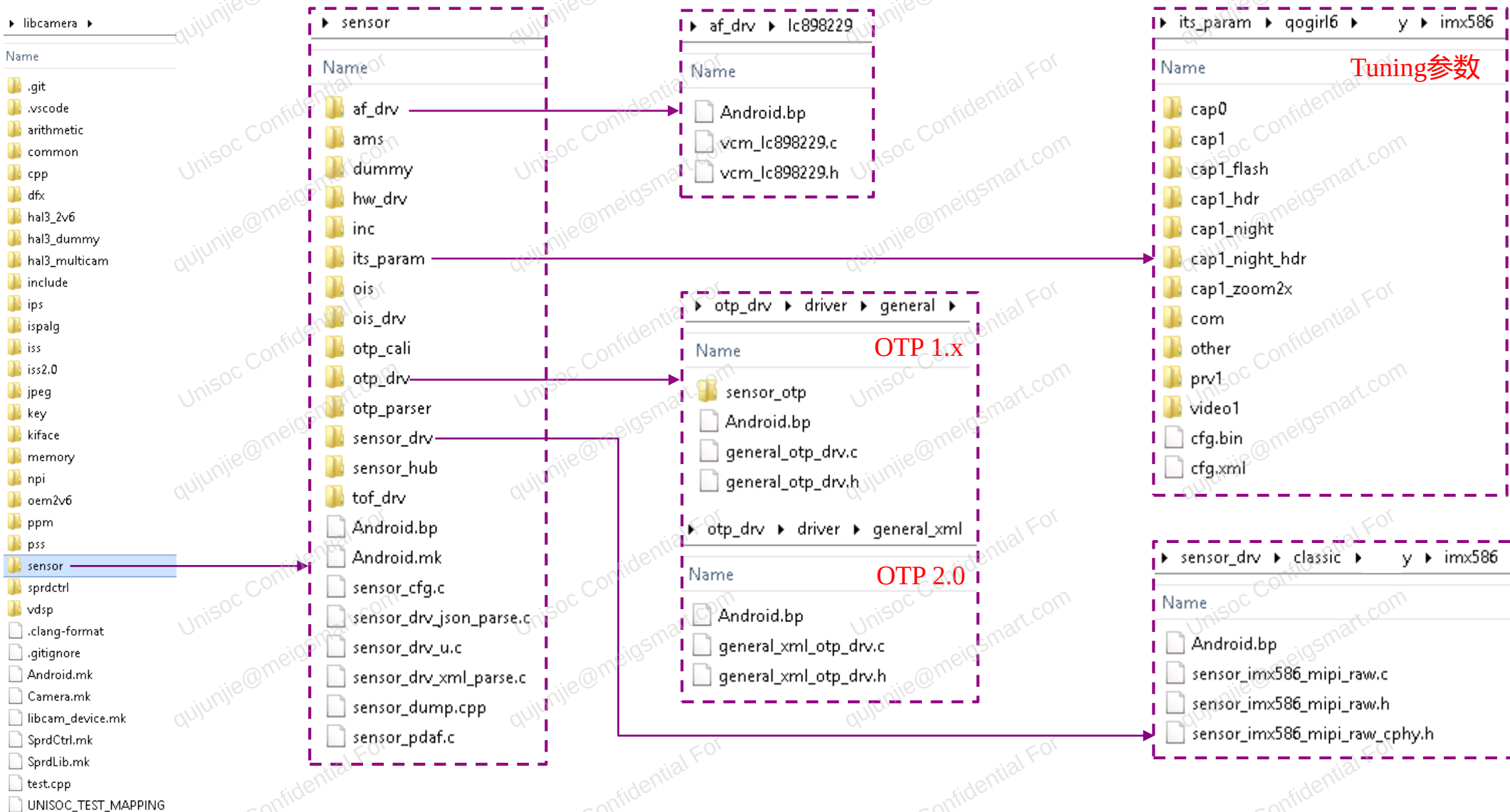
01

Camera驱动 代码结构



Camera驱动代码结构

本文以UMS9230或UMS512平台为例说明，libcamera路径：sprdroid13_main/vendor/sprd/modules/libcamera/。



02

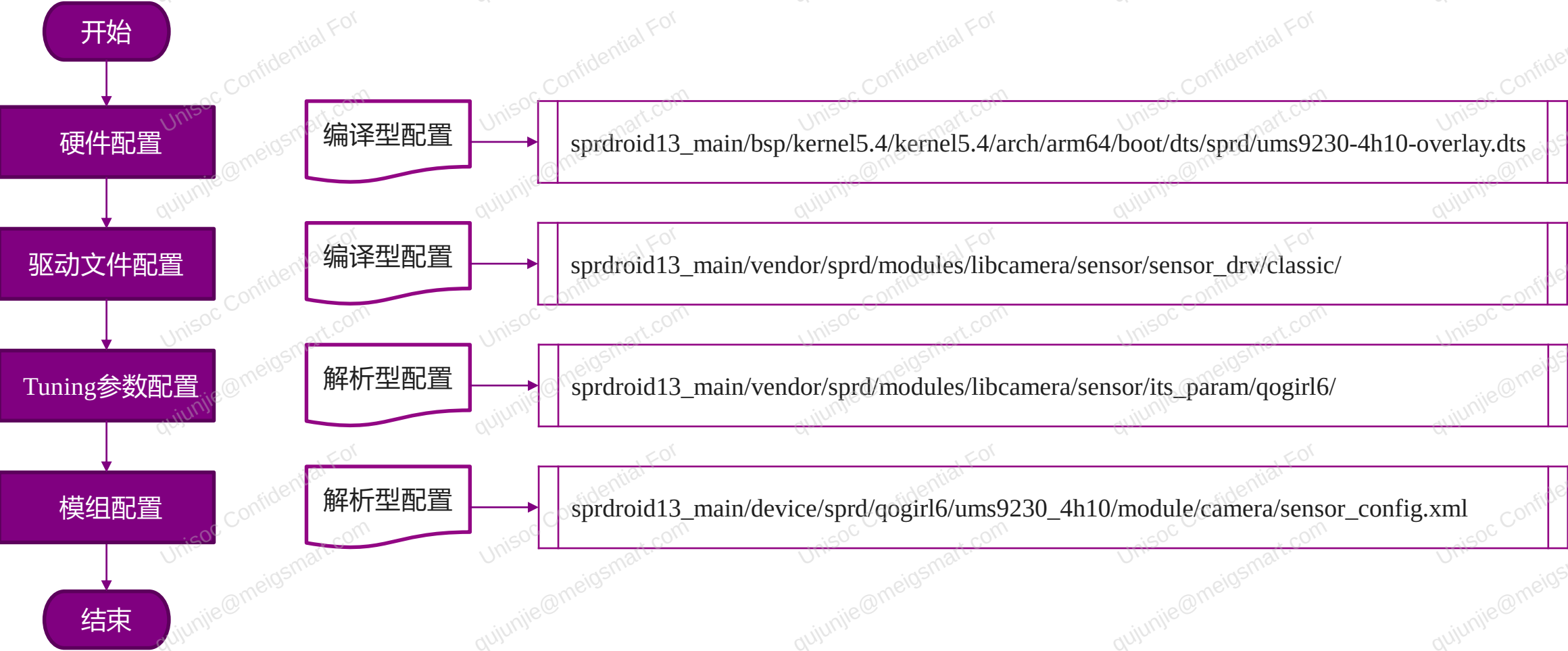
Sensor驱动 配置 配置流程



配置流程



以board ums9230_4h10为例。



02

Sensor驱动 配置 硬件配置



DTS文件为sprdroid13_main/bsp/kernel5.4/kernel5.4/arch/arm64/boot/dts/sprd/ums9230-4h10-overlay.dts，根据实际使用平台的硬件原理图配置。

I2C总线

```
&i2c0 {
```

```
    status = "okay";  
    clock-frequency = <4000000>;  
    #address-cells = <1>;  
    #size-cells = <0>;
```

```
sensor_main: sensor-main@20 {  
    compatible = "sprd,sensor-main";  
    reg = <0x20>;
```

```
    clock-names = "clk_src", "sensor_eb",  
                  "clk_96m", "clk_76m8",  
                  "clk_48m", "clk_26m";
```

```
    clocks = <&mm_clk CLK_SENSOR0>, <&mm_gate CLK_MM_SENSOR0_EB>,  
             <&p111 CLK_TWPLL_96M>, <&p111 CLK_TWPLL_76M8>,  
             <&p111 CLK_TWPLL_48M>, <&ext_26m>;
```

```
    vddio-supply = <&vddcamio>;  
    vddcama-supply = <&vddcama0>;  
    dvdd-gpios = <&ap_gpio 137 0>;  
    vddcamnot-supply = <&vddcamnot>;  
    reset-gpios = <&ap_gpio 45 0>;  
    power-down-gpios = <&ap_gpio 47 0>;  
    sprd,phyid = <0>;  
    csi = <&csi0>;  
    power-domains = <&mm_domain>;
```

```
};
```

Sensor ID

- sensor_main: sensor_id 0 (一般为后主摄)
- sensor_sub: sensor_id 1 (一般为前摄)
- sensor_main2: sensor_id 2 (一般为后辅摄)
- sensor_sub2: sensor_id 3
- sensor_main3: sensor_id 4
- sensor_sub3: sensor_id 5

I2C地址

非实际使用I2C地址，用来区分I2C设备（相同I2C总线内，不同I2C设备此地址不能相同），建议填写实际I2C地址。

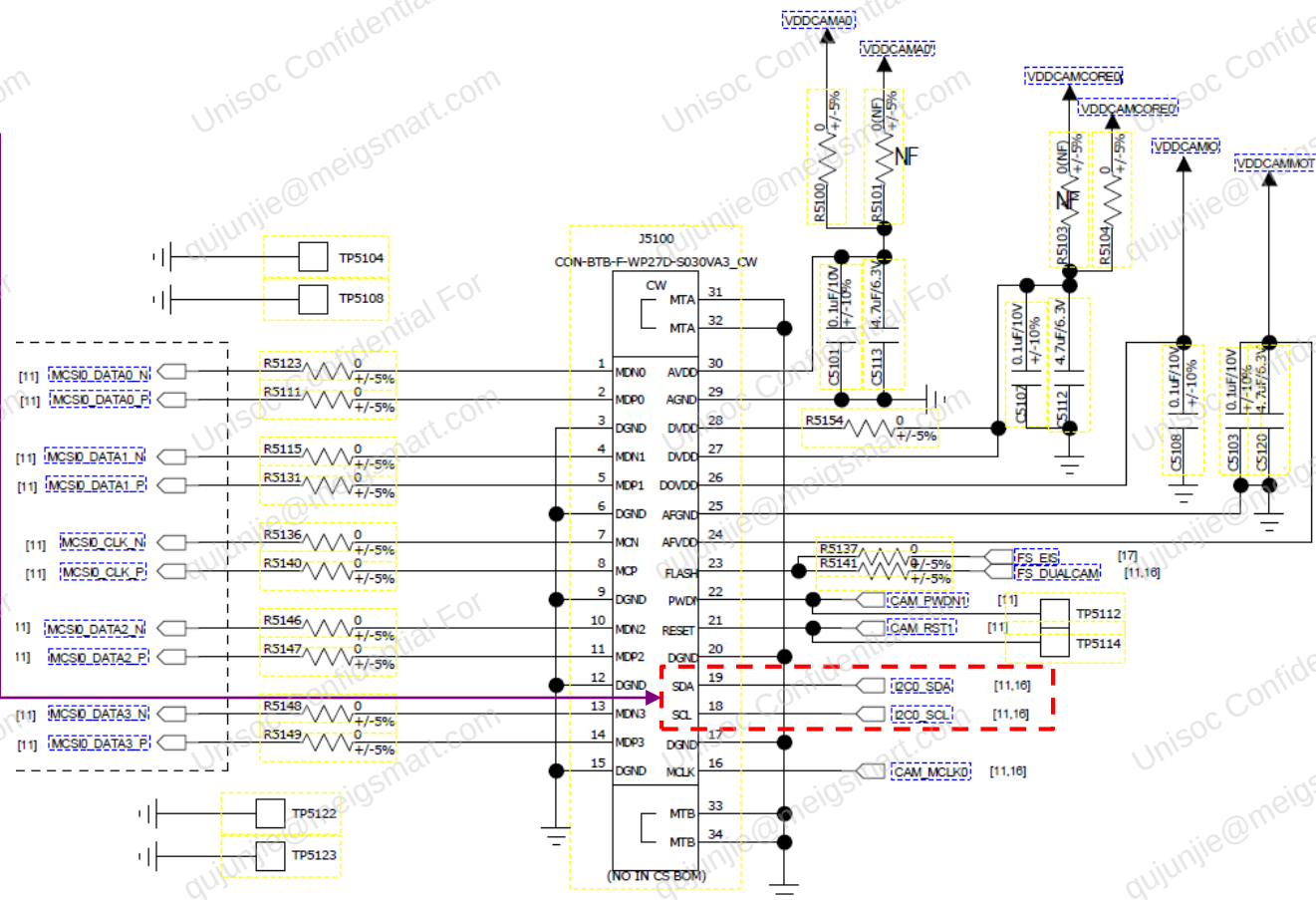
依次为IOVDD、
AVDD、DVDD和
AFVDD（马达）

RESET和PWDN

PHY和CSI

} :

1:



供电总览

确定供电配置。以ums9230_4h10后主摄为例，查看接通的电源引脚对应的名称，NF表示未接通。其中IOVDD、AVDD和AFVDD（马达）均为电源直接供电，DVDD为GPIO控制LDO供电。

```
&i2c0 {
    status = "okay";
    clock-frequency = <400000>;
    #address-cells = <1>;
    #size-cells = <0>;

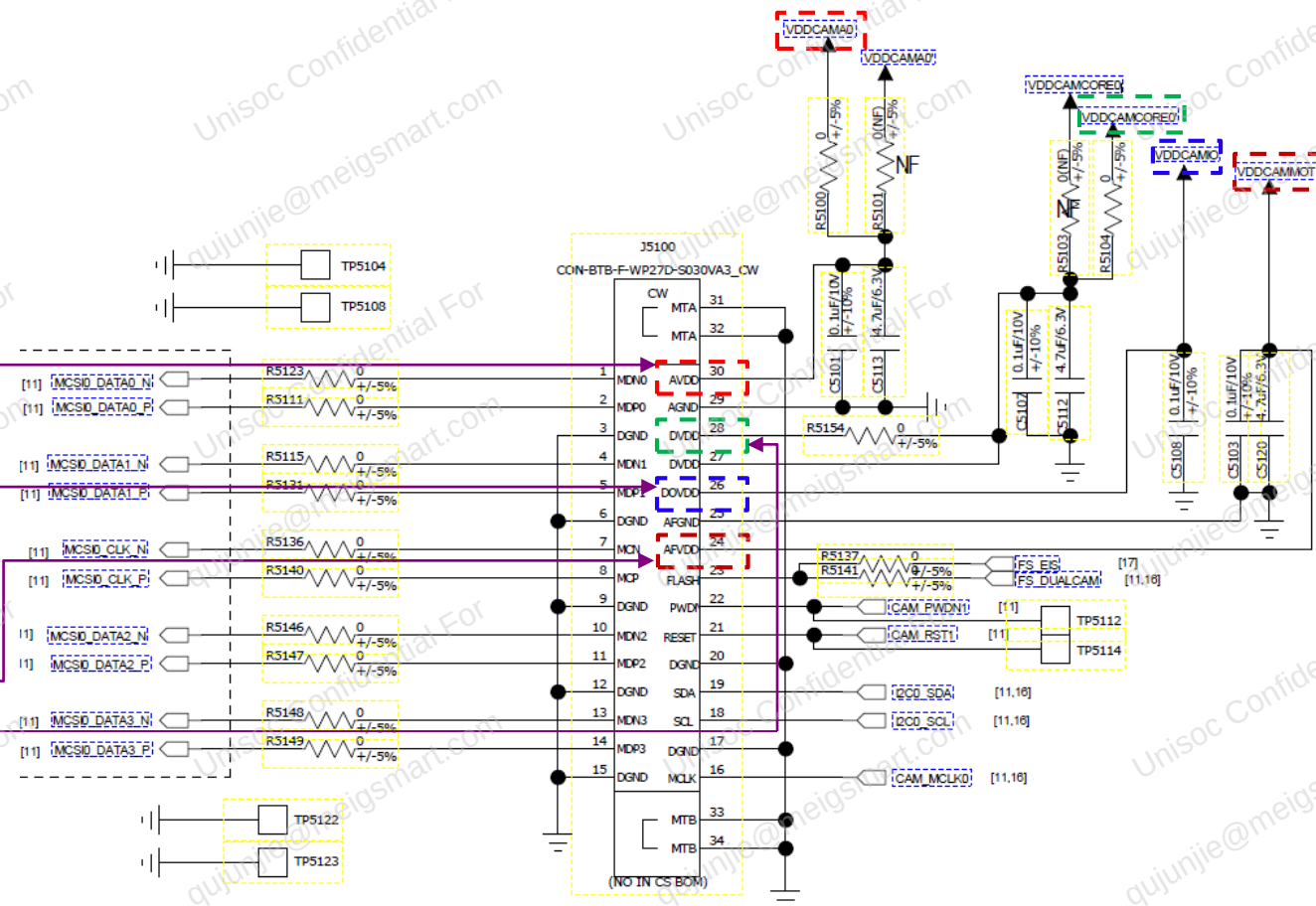
    sensor_main: sensor-main@20 {
        compatible = "sprd,sensor-main";
        reg = <0x20>;
        clock-names = "clk_src", "sensor_eb",
            "clk_96m", "clk_76m8",
            "clk_48m", "clk_26m";
        clocks = <&mm_clk CLK_SENSORU>, <&mm_gate CLK_MM_SENSORU_EB>,
            <&p111 CLK_TWPLL_96M>, <&p111 CLK_TWPLL_76M8>,
            <&p111 CLK_TWPLL_48M>, <&ext_26m>;
        vddio-supply = <&vddcamio>;
        vddcama-supply = <&vddcama0>;
        dvdd-gpios = <&ap_gpio 137 0>;
        vddcamnot-supply = <&vddcamnot>;
        reset-gpios = <&ap_gpio 45 0>;
        power-down-gpios = <&ap_gpio 47 0>;
        sprd,phyid = <0>;
        csi = <&csi0>;
        power-domains = <&mm_domain>;
    };
};
```

AVDD

IOVDD

AFVDD (马达)

DVDD



IOVDD、AVDD和AFVDD (马达)

IOVDD、AVDD和AFVDD (马达) 为电源直接供电, 由Camera硬件原理图获知, 对应引脚依次为VDDCAMIO、VDDCAMA0和VDDCAMMOT。与电源硬件原理图对应关系如下:

```
&i2c0 {  
    status = "okay";  
    clock-frequency = <400000>;  
    #address-cells = <1>;  
    #size-cells = <0>;
```

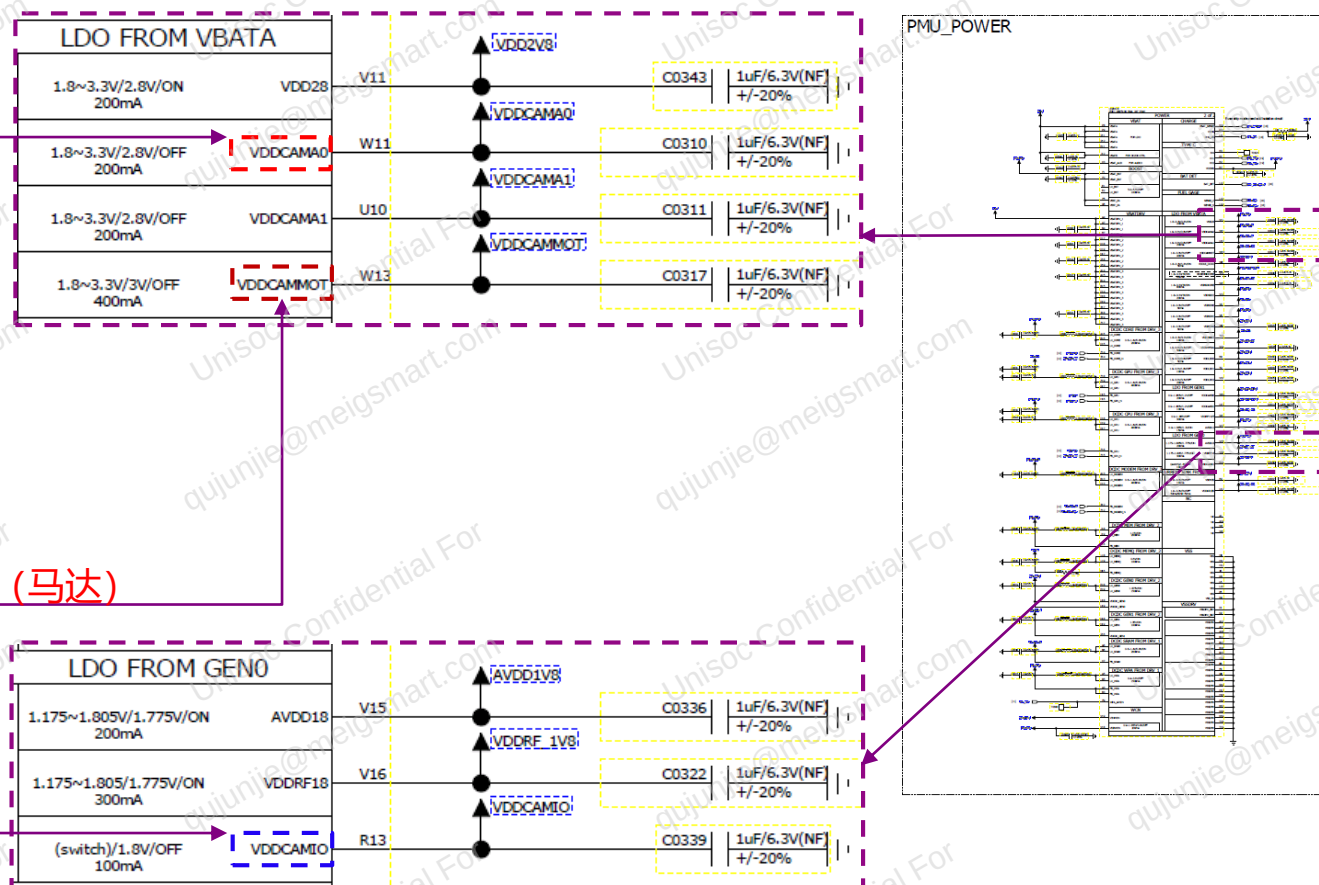
```
sensor_main: sensor-main@20 {  
    compatible = "sprd,sensor-main";  
    reg = <0x20>;  
    clock-names = "clk_src", "sensor_eb",  
        "clk_96m", "clk_76m8",  
        "clk_48m", "clk_26m";  
    clocks = <&mm_clk CLK_SENSOR0>, <&mm_gate CLK_MM_SENSOR0_EB>,  
        <&p111 CLK_TWPLL_96M>, <&p111 CLK_TWPLL_76M8>,  
        <&p111 CLK_TWPLL_48M>, <&ext_26m>;
```

```
    vddio-supply = <&vddcamio>;  
    vddcama-supply = <&vddcama0>;  
    dvdd-gpios = <&ap_gpio 137 0>;  
    vddcammot-supply = <&vddcammot>;  
    reset-gpios = <&ap_gpio 45 0>;  
    power-down-gpios = <&ap_gpio 47 0>;  
    sprd,phyid = <0>;  
    csi = <&csi0>;  
    power-domains = <&mm_domain>;  
};
```

AVDD

AFVDD (马达)

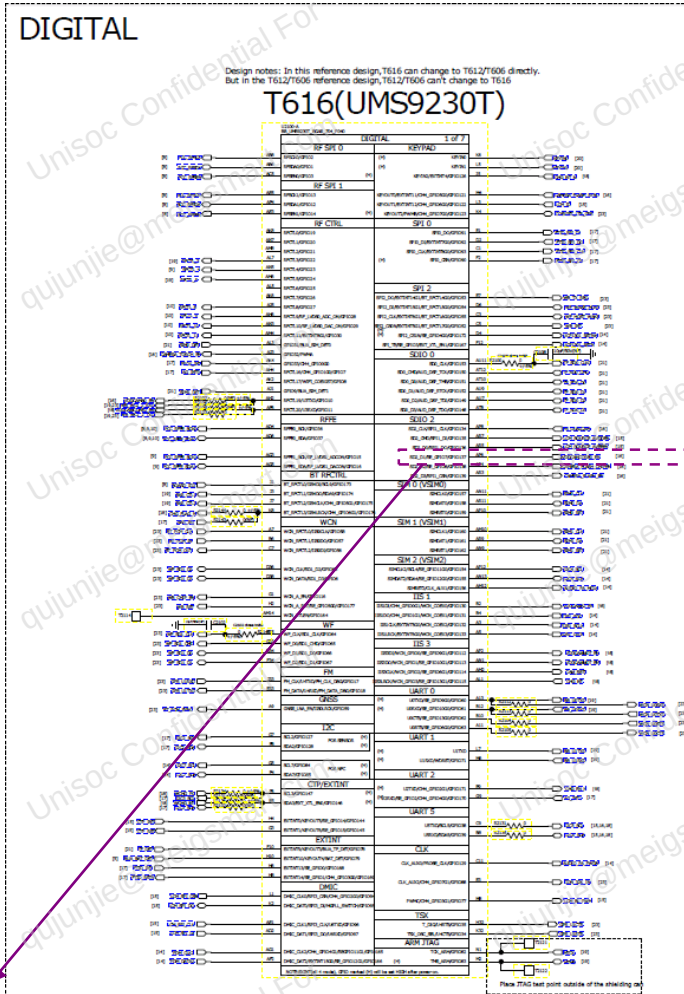
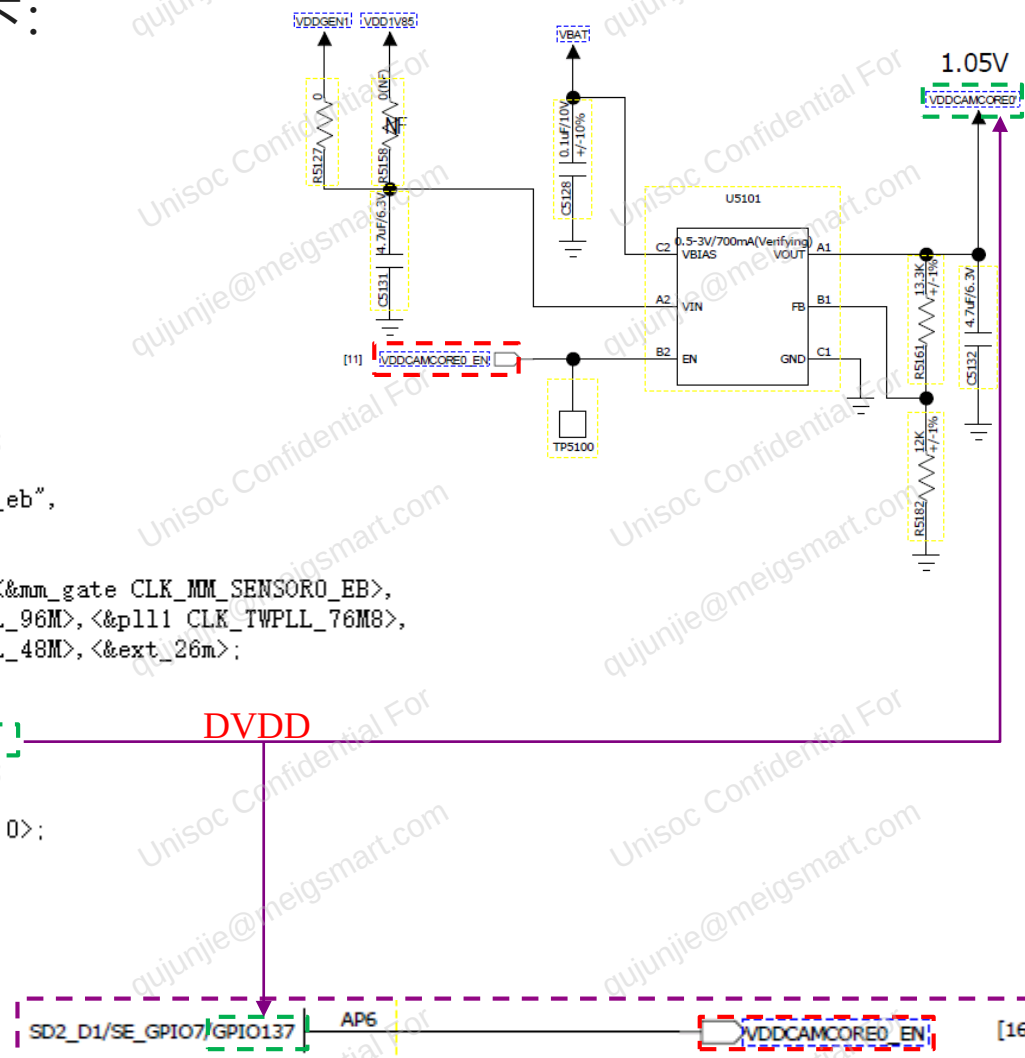
IOVDD



 紫光展锐
UNISOC

```
&i2c0 {
    status = "okay";
    clock-frequency = <400000>;
    #address-cells = <1>;
    #size-cells = <0>;

    sensor_main: sensor-main@20 {
        compatible = "sprd,sensor-main";
        reg = <0x20>;
        clock-names = "clk_src", "sensor_eb",
            "clk_96m", "clk_76m8",
            "clk_48m", "clk_26m";
        clocks = <&mm_clk CLK_SENSOR0>, <&mm_clk CLK_SENSOR1>,
            <&p111 CLK_TWPLL_96>,
            <&p111 CLK_TWPLL_48>;
        vddio-supply = <&vddcamio>;
        vddcam-supply = <&vddcam0>;
        dvdd-gpios = <&ap_gpio 137 0>;
        vddcamnot-supply = <&vddcamnot>;
        reset-gpios = <&ap_gpio 45 0>;
        power-down-gpios = <&ap_gpio 47 0>;
        sprd,phyid = <0>;
        csi = <&csi0>;
        power-domains = <&mm_domain>;
    };
};
```



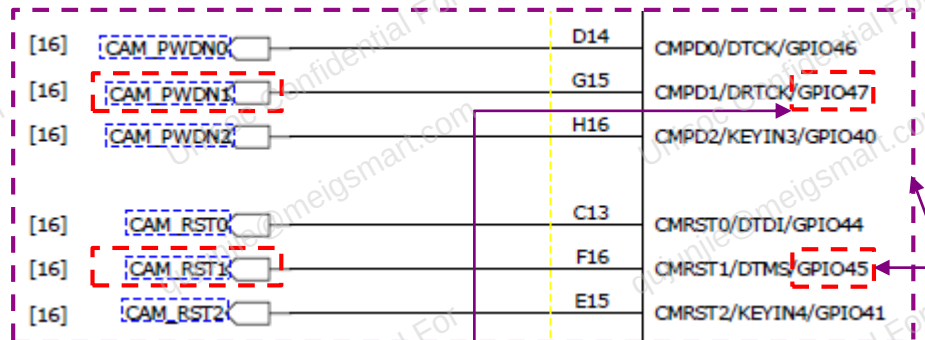
RESET和PWDN

以ums9230_4h10后主摄为例，RESET和PWDN为GPIO控制，由Camera硬件原理图获知，对应引脚依次为CAM_RST1和CAM_PWDN1。与GPIO对应关系如下：

```
&i2c0 {  
    status = "okay";  
    clock-frequency = <400000>;  
    #address-cells = <1>;  
    #size-cells = <0>;
```

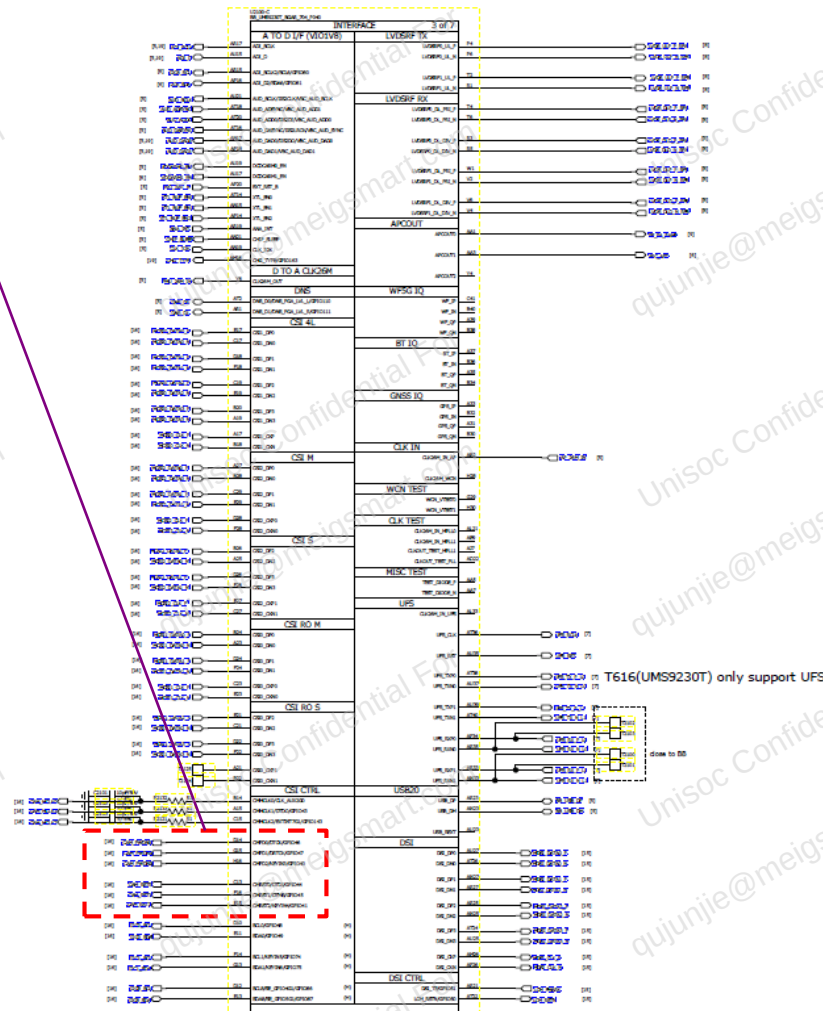
```
sensor_main: sensor-main@20 {  
    compatible = "sprd,sensor-main";  
    reg = <0x20>;  
    clock-names = "clk_src","sensor_eb",  
        "clk_96m","clk_76m8",  
        "clk_48m","clk_26m";  
    clocks = <&mm_clk CLK_SENSOR0>,<&mm_gate CLK_MM_SENSOR0_EB>,  
        <&p111 CLK_TWPLL_96M>,<&p111 CLK_TWPLL_76M8>,  
        <&p111 CLK_TWPLL_48M>,<&ext_26m>;  
    vddio-supply = <&vddcamio>;  
    vddcama-supply = <&vddcama0>;  
    dvdd-gpios = <&ap_gpio 137 0>;  
    vddcamnot-supply = <&vddcamnot>;  
    reset-gpios = <&ap_gpio 45 0>;  
    power-down-gpios = <&ap_gpio 47 0>;  
    sprd,phyid = <0>;  
    csi = <&csi0>;  
    power-domains = <&mm_domain>;
```

```
};
```



PWDN

RESET



PHY和CSI 1/2

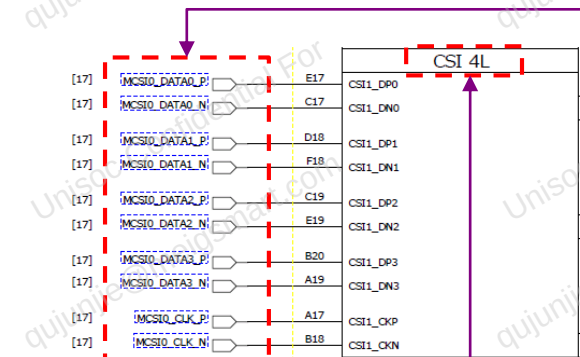
确定PHY配置。以ums9230_4h10后主摄为例，由Camera硬件原理图获知，PHY为4lane。

```
&i2c0 {
    status = "okay";
    clock-frequency = <400000>;
    #address-cells = <1>;
    #size-cells = <0>;

    sensor_main: sensor-main@20 {
        compatible = "sprd,sensor-main";
        reg = <0x20>;
        clock-names = "clk_src","sensor_eb",
            "clk_96m","clk_76m8",
            "clk_48m","clk_26m";
        clocks = <&mm_clk CLK_SENSOR0>,<&mm_gate CLK_MM_SENSOR0_EB>,
            <&p111 CLK_TWPLL_96M>,<&p111 CLK_TWPLL_76M8>,
            <&p111 CLK_TWPLL_48M>,<&ext_26m>;
        vddio-supply = <&vddcamio>;
        vddcama-supply = <&vddcama0>;
        dvdd-gpios = <&ap_gpio 137 0>;
        vddcamnot-supply = <&vddcamnot>;
        reset-gpios = <&ap_gpio 45 0>;
        power-down-gpios = <&ap_gpio 47 0>;
        sprd,phyid = <0>;
        csi = <&csi0>;
        power-domains = <&mm_domain>;
    };

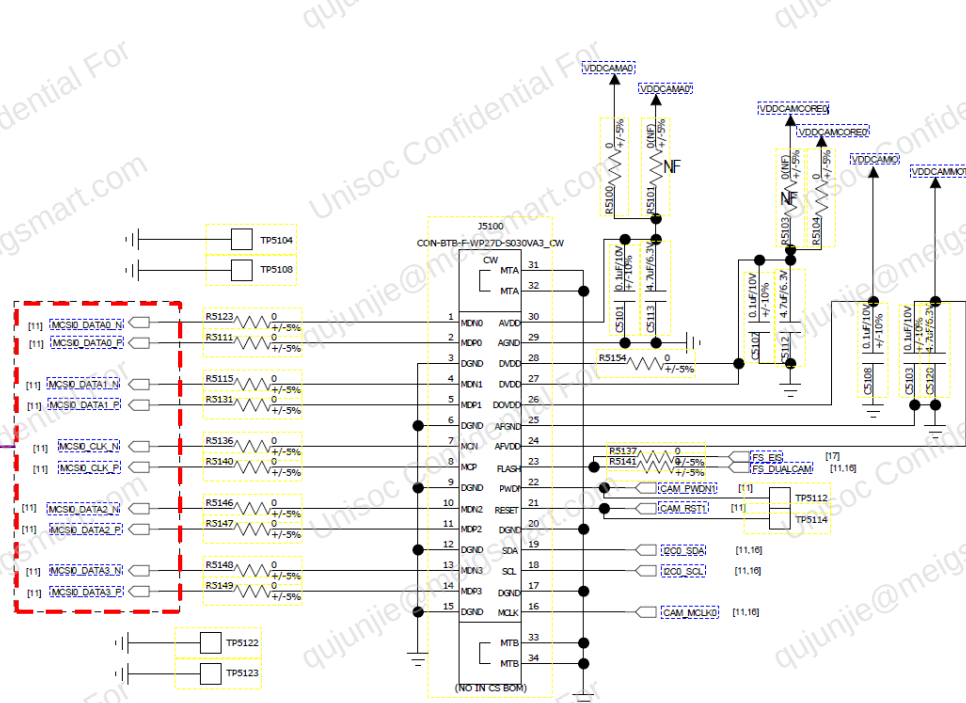
    mipi_csi_phy0: mipi-csi-phy0 {
        compatible = "sprd,mipi-csi-phy";
        sprd,phyid = <0>;
        sprd,phynum = "4lane";
        status = "disabled";
    };
};
```

qogirl6.dtsi



```
enum csi_phy_t {
    PHY_4LANE = 0, //phy_0
    PHY_2P2, //phy_1
    PHY_2P2_M,
    PHY_2P2_S,
    PHY_2P2RO_M,
    PHY_2P2RO_S,
    PHY_MAX,
};
```

bsp/modules/common/camera/sensor/csi2/sprd/receiver_r3p1/csi_driver.h

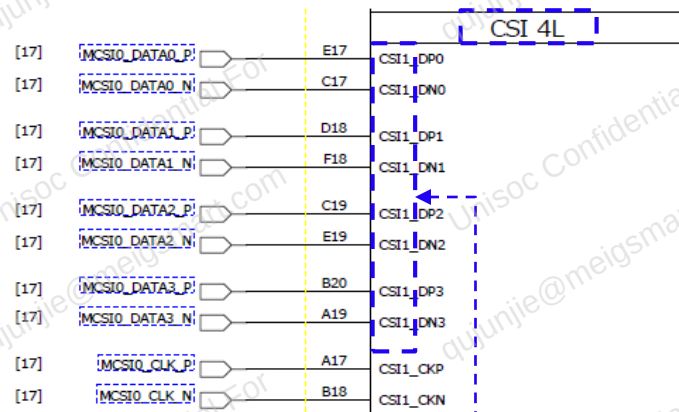


- DTS中的PHY编号，配置的是硬件原理图中的CSI接口，PHY编号（0/1/2等）与CSI接口（4lane/2p2/2p2_m/2p2_s/2p2ro/2p2ro_s）的对应关系，可在对应平台的dtsi文件中查询。
- DTS中的PHY和CSI编号，由软件自定义。

确定CSI配置。以ums9230_4h10后主摄为例，软件定义CSI0对应PHY为4lane（对应dcam0）。

```
&i2c0 {
    status = "okay";
    clock-frequency = <400000>;
    #address-cells = <1>;
    #size-cells = <0>;

    sensor_main: sensor-main@20 {
        compatible = "sprd,sensor-main";
        reg = <0x20>;
        clock-names = "clk_src","sensor_eb",
            "clk_96m","clk_76m8",
            "clk_48m","clk_26m";
        clocks = <&mm_clk CLK_SENSOR0>,<&mm_gate CLK_MM_SENSOR0_EB>,
            <&p111 CLK_TWPLL_96M>,<&p111 CLK_TWPLL_76M8>,
            <&p111 CLK_TWPLL_48M>,<&ext_26m>;
        vddio-supply = <&vddcamio>;
        vddcam-supply = <&vddcam0>;
        dvdd-gpios = <&ap_gpio 137 0>;
        vddcamnot-supply = <&vddcamnot>;
        reset-gpios = <&ap_gpio 45 0>;
        power-down-gpios = <&ap_gpio 47 0>;
        sprd,phyid = <0>;
        csi = <&csi0>;
        power-domains = <&mm_domain>;
    };
};
```



```
enum csi_controller_t {
    CSI_RX0 = 0, //csi 4l (csi 1)
    CSI_RX1, //csi m/s (csi 2)
    CSI_RX2, //csi ro m/s (csi 0)
    CSI_MAX_COUNT,
};

enum csi_phy_t {
    PHY_4LANE = 0, //phy_0
    PHY_2P2, //phy_1
    PHY_2P2R0, //phy_2
    PHY_2P2R1,
    PHY_2P2R2,
    PHY_2P2R3,
    PHY_MAX,
};
```

.../csi2/sprd/receiver_r3p1/csi_driver.h

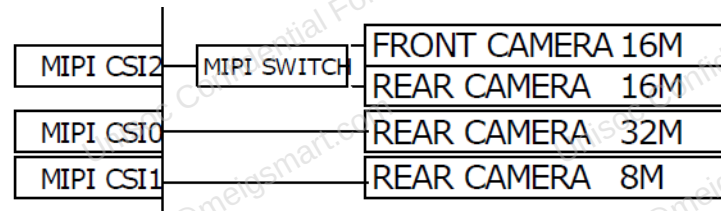
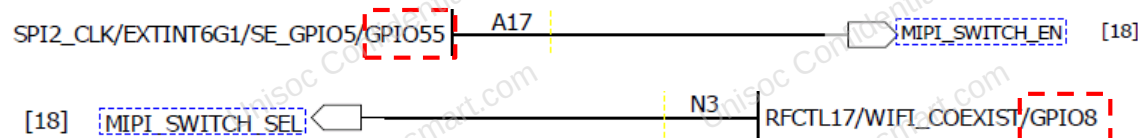
```
csi0: csi00@3e200000 {
    compatible = "sprd,csi-controller";
    reg = <0 0x3e200000 0 0x1000>;
    interrupts = <GIC_SPI 45 IRQ_TYPE_LEVEL_HIGH>,
        <GIC_SPI 46 IRQ_TYPE_LEVEL_HIGH>,
        <GIC_SPI 47 IRQ_TYPE_LEVEL_HIGH>,
        <GIC_SPI 48 IRQ_TYPE_LEVEL_HIGH>;
    clock-names = "clk_mipi_csi_gate_eb",
        "clk_csi_eb",
        "mipi_csi_src_eb";
    clocks = <&mm_gate CLK_MM_MIPI_CSI0_EB>,
        <&mm_gate CLK_MM_CSI0_EB>,
        <&mm_clk CLK_MIPI_CSI0>;
    sprd,aon-apb-syscon = <&aon_apb_regs>;
    sprd,cam-ahb-syscon = <&mm_ahb_regs>;
    sprd,anlg_phy_g10_controller = <&anlg_phy_g10_regs>;
    sprd,csi-id = <0>;
    sprd,dcam-id = <0>;
    sprd,ip-version = <0x200>;
    status = "okay";
};
```

```
mipi_csi_phy0: mipi-csi-phy0 {
    compatible = "sprd,mipi-csi-phy";
    sprd,phyid = <0>;
    sprd,phyname = "4lane";
    status = "disabled";
};
```

qogirl6.dtsi

MIPI SWITCH

以ums512_1h10前摄和后辅摄（均为4lane）为例（UMS9230没启用MIPI SWITCH），文件为sprdroid13_main/bsp/kernel5.4/kernel5.4/arch/arm64/boot/dts/sprd/ums512-1h10-overlay.dts。



```
sensor_sub: sensor-sub@5a {
    compatible = "sprd,sensor-sub";
    reg = <0x5a>;
    clock-names = "clk_src", "sensor_eb",
        "clk_96m", "clk_76m8",
        "clk_48m", "clk_26m";
    clocks = <&mm_clk CLK_SENSOR0>, <&mm_gate CLK_MM_SENSOR0_EB>,
        <&p112 CLK_TWPLL_96M>, <&p112 CLK_TWPLL_76M8>,
        <&p112 CLK_TWPLL_48M>, <&ext_26m>;
    vddio-supply = <&vddcamio>;
    vddcama-supply = <&vddcamal>;
    vddcamd-supply = <&vddcamdl>;
    vddcamnot-supply = <&vddcamnot>;
    reset-gpios = <&ap_gpio 41 0>;
    power-down-gpios = <&ap_gpio 40 0>;
    mipi-switch-en-gpios = <&ap_gpio 55 0>;
    mipi-switch-mode-gpios = <&ap_gpio 8 0>;
    sprd,phyid = <1>;
    csi = <&csi1>;
    power-domains = <&mm_domain>;
};
```

在Sensor驱动中控制

共用同一PHY和CSI

```
sensor_main2: sensor-main2@20 {
    compatible = "sprd,sensor-main2";
    reg = <0x20>;
    clock-names = "clk_src", "sensor_eb",
        "clk_96m", "clk_76m8",
        "clk_48m", "clk_26m";
    clocks = <&mm_clk CLK_SENSOR1>, <&mm_gate CLK_MM_SENSOR1_EB>,
        <&p112 CLK_TWPLL_96M>, <&p112 CLK_TWPLL_76M8>,
        <&p112 CLK_TWPLL_48M>, <&ext_26m>;
    vddio-supply = <&vddcamio>;
    vddcama-supply = <&vddcamal>;
    vddcamd-supply = <&vddcamdl>;
    vddcamnot-supply = <&vddcamnot>;
    avdd-gpios = <&ap_gpio 33 0>;
    mot-gpios = <&ap_gpio 88 0>;
    reset-gpios = <&ap_gpio 45 0>;
    power-down-gpios = <&ap_gpio 47 0>;
    mipi-switch-en-gpios = <&ap_gpio 55 0>;
    mipi-switch-mode-gpios = <&ap_gpio 8 0>;
    sprd,phyid = <1>;
    csi = <&csi1>;
    power-domains = <&mm_domain>;
};
```

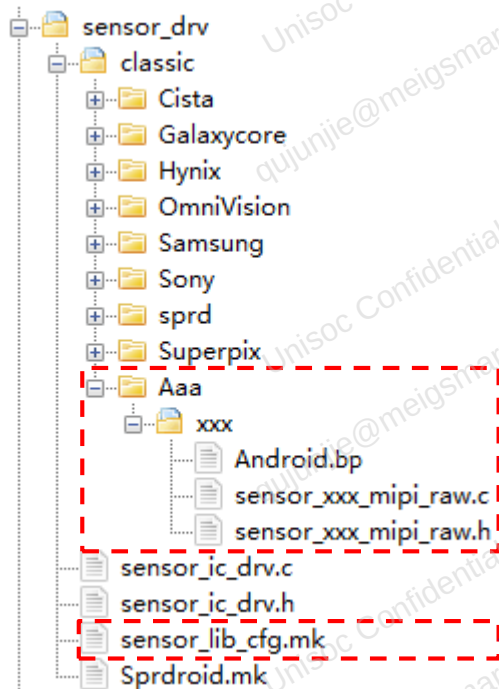
02

Sensor驱动配置

驱动文件配置

驱动文件

路径为sprdroid13_main/vendor/sprd/modules/libcamera/sensor/sensor_drv/。



添加驱动文件夹

- Aaa: Sensor Vendor
- xxx: Sensor Name, 需与sensor_config.xml中SensorName同名, 驱动文件请参照工程自带驱动添加

将so打包到pac

```
PRODUCT_PACKAGES += libsensor_imx351 \  
libsensor_ov5675_dual \  
libsensor_ov8856_shine \  
libsensor_xxx
```


- 平台支持PDAF（Phase Detection Auto Focus，相位检测自动对焦）类型及说明如下表。

PDAF类型	相位计算	配置内容
Type1	Sensor计算	相位差数据传输方式
Type2/Dual PD	Sensor提取PD数据，平台计算	PD数据传输方式，PD点位置信息
Type3	平台提取PD数据并计算	PD点位置信息

- 涉及文件为sensor_xxx_mipi_raw.h和sensor_xxx_mipi_raw.c。

```
static SENSOR_STATIC_INFO_T s_xxx_static_info[VENDOR_NUM] = {  
    {.module_id = MODULE_YYY,  
     .static_info = {.f_num = 200,  
                     .focal_length = 352,  
                     .max_fps = 0,  
                     .max_adgain = 16,  
                     .ois_supported = 0,  
                     .pda_f_supported = SENSOR_PDAF_TYPE1_ENABLE,  
                     .embedded_line_enable = 0,  
                     .exp_valid_frame_num = 1,  
                     .clamp_level = 64,  
                     .adgain_valid_frame_num = 2,  
                     .fov_info = {{4.656f, 3.496f}, 3.698f},  
                     .mono_sensor = 0}}  
};
```

sensor_xxx_mipi_raw.h

配置PDAF类型

- SENSOR_PDAF_TYPE1_ENABLE
- SENSOR_PDAF_TYPE2_ENABLE
- SENSOR_PDAF_TYPE3_ENABLE
- SENSOR_DUAL_PDAF_ENABLE

需实现接口，上传PDAF配置信息

```
static cmr_int xxx_drv_access_val(cmr_handle handle, cmr_uint param)  
{  
    .....  
    switch(param_ptr->type)  
    {  
        .....  
        case SENSOR_VAL_TYPE_GET_PDAF_INFO:  
            ret = xxx_drv_get_pda_f_info(handle, param_ptr->pval);  
            break;  
    }
```

sensor_xxx_mipi_raw.c

PDAF 2/5

文件: sensor_xxx_mipi_raw.c

函数: static cmr_int xxx_drv_get_pdaf_info(cmr_handle handle, cmr_u32 *param)

通过Virtual Channel(VC)方式传输PD数据

```
pdaf_info->vch2_info.bypass = 0;  
pdaf_info->vch2_info.vch2_vc = 1;  
pdaf_info->vch2_info.vch2_data_type = 0;  
pdaf_info->vch2_info.vch2_mode = 0x02;  
pdaf_info->sns_mode = pd_sns_mode;
```

选择VC或DT

VC或DT配置
Type3配置为3

配置了PDAF的
sensor setting

通过Data Type(DT)方式传输PD数据

```
pdaf_info->vch2_info.bypass = 0;  
pdaf_info->vch2_info.vch2_vc = 0;  
pdaf_info->vch2_info.vch2_data_type = 0x19;  
pdaf_info->vch2_info.vch2_mode = 0x01;  
pdaf_info->sns_mode = pd_sns_mode;
```

Sensor的data type, 可参考Sensor spec

```
static const cmr_u32 pd_sns_mode[] = {0, 0, 1, 1};
```

未配置PDAF填写0, 配置PDAF填写1, 参照Sensor头文件 sensor_imx586_mipi_raw.h中s_imx586_resolution_tab_raw结构体 setting顺序配置, 图示为展锐工程模板示例:

1. xxx_init_setting: 未配置
2. xxx_video_setting: 未配置
3. xxx_preview_setting: 已配置
4. xxx_snapshot_setting: 已配置

文件: sensor_xxx_mipi_raw.c

函数: static cmr_int xxx_drv_get_pdaf_info(cmr_handle handle, cmr_u32 *param)

PD点位置信息配置

遮住右半边表示右点, 用1表示
遮住左半边表示左点, 用0表示

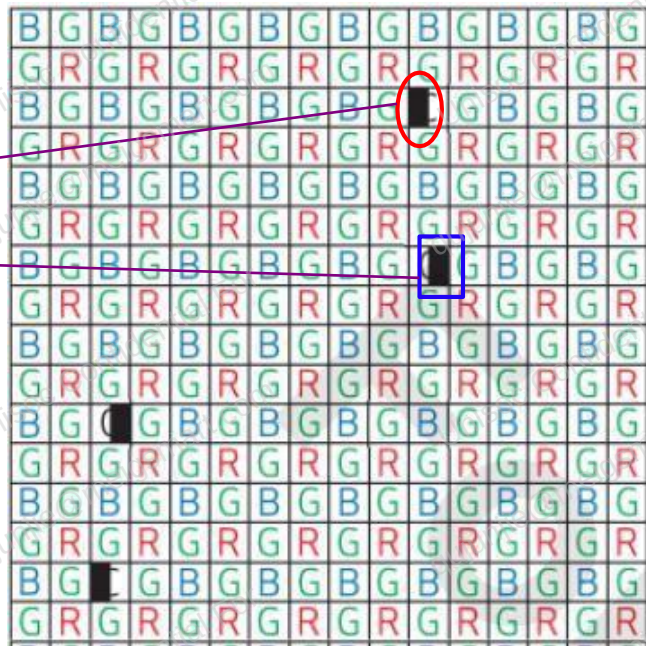
```
static const cmr_u16 xxx_pd_is_right[] = {0, 1, 1, 0};
```

PD点行和列的索引从0开始

```
static const cmr_u16 xxx_pd_row[] = {2, 6, 10, 14};
static const cmr_u16 xxx_pd_col[] = {10, 10, 2, 2};
```

左右PD点的坐标

```
static const struct pd_pos_info xxx_pd_pos_l[] = {
    {10, 2}, {2, 14},
};
static const struct pd_pos_info xxx_pd_pos_r[] = {
    {10, 6}, {2, 10},
};
```



展锐平台对于左右PD点的定义

- Shield PD类型: 遮左半边pixel定义为左点, 遮右半边pixel定义为右点。
- Super PD类型: 由于是2x1 ocl结构, 固定为两个相邻PD pixel组合, 左边的pixel为右点, 右边的pixel为左点。

文件: sensor_xxx_mipi_raw.c

函数: static cmr_int xxx_drv_get_pdaf_info(cmr_handle handle, cmr_u32 *param)

```
static cmr_int xxx_drv_get_pdaf_info(cmr_handle handle, cmr_u32 *param)
{
    cmr_int rtn = SENSOR_SUCCESS;
    struct sensor_pdaf_info *pdaf_info = NULL;
    SENSOR_IC_CHECK_PTR(param);
    cmr_u16 i = 0;
    cmr_u16 pd_pos_row_size = 0;
    cmr_u16 pd_pos_col_size = 0;
    cmr_u16 pd_pos_is_right_size = 0;

    pdaf_info = (struct sensor_pdaf_info *)param;
    pd_pos_is_right_size = NUMBER_OF_ARRAY(xxx_pd_is_right);
    pd_pos_row_size = NUMBER_OF_ARRAY(xxx_pd_row);
    pd_pos_col_size = NUMBER_OF_ARRAY(xxx_pd_col);
    if ((pd_pos_row_size != pd_pos_col_size) ||
        (pd_pos_row_size != pd_pos_is_right_size) ||
        (pd_pos_is_right_size != pd_pos_col_size)) {
        SENSOR_LOGE("pd_pos_row size and pd_pos_is_right size are not match");
        return SENSOR_FAIL;
    }

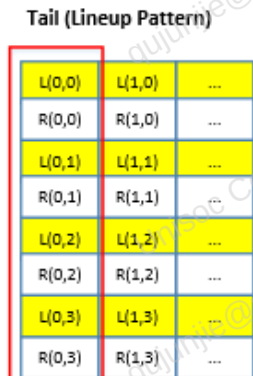
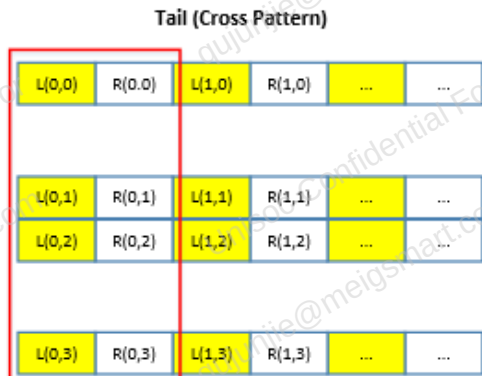
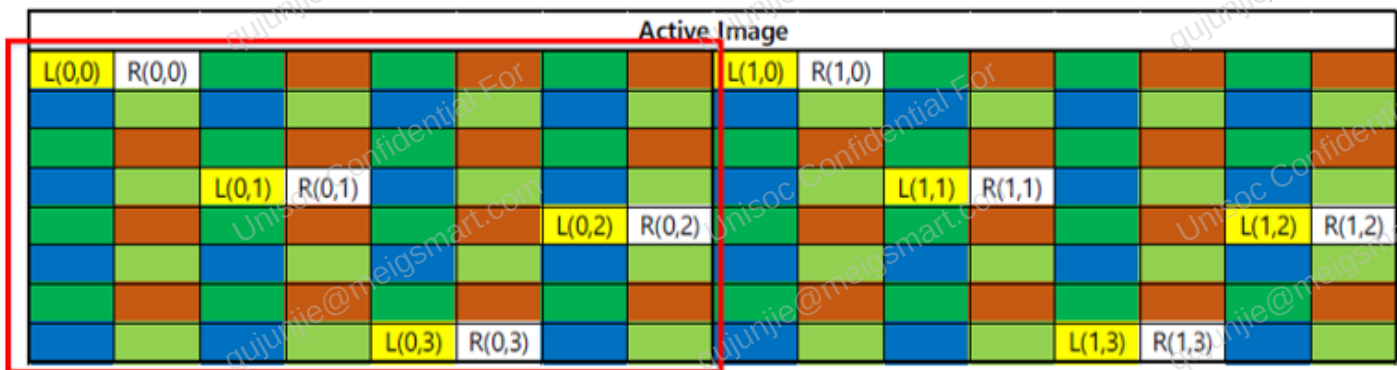
    pdaf_info->pd_offset_x = 0; //horizontal position of first PD pattern
    pdaf_info->pd_offset_y = 0; //vertical position of first PD pattern
    pdaf_info->pd_end_x = 4672; //horizontal position of last PD pattern
    pdaf_info->pd_end_y = 3504; //vertical position of last PD pattern
    pdaf_info->pd_density_x = 16; //horizontal distance between PD pairs
    pdaf_info->pd_density_y = 8; //vertical distance between PD pairs
    pdaf_info->pd_block_w = 1; //width of basic pd pattern:0:8;1:16;2:32;3:64;max:3
    pdaf_info->pd_block_h = 1; //height of basic pd pattern:0:8;1:16;2:32;3:64;max:3
    pdaf_info->pd_block_num_x = 292; // horizontal number of basic pd pattern
    pdaf_info->pd_block_num_y = 219; // vertical number of basic pd pattern
    pdaf_info->vendor_type = SENSOR_VENDOR_XXX; //please with unisoc
```

配置示例

文件: sensor_xxx_mipi_raw.c

Type2 sensor抽取PD左右点方式配置 (以8 × 8 pd_block举例) 。

```
struct pdaf_block_descriptor xxx_pd_seperator_helper = {
    .block_width = 2, //Tail mode output width per pd block.
    .block_height = 4, //Tail mode output height per pd block.
    .line_width = 2, //CROSS_PATTERN only. Equals to block_width.
    .block_pattern = CROSS_PATTERN, //Tail mode pattern. CROSS_PATTERN or LINE_UP
    .pd_line_coordinate = imx586_pd_coordinate_table, //Unused. Duplicated with xxx_pd_is_right.
    .coordinate_tab = {0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1},
    //Lineup pattern only. Tail mode output coordinate.
    //0 refers to left pd pixel line; 1 refers to right pd pixel line.
};
```



Group hold

文件: sensor_xxx_mipi_raw.h

```
static SENSOR_REG_T xxx_grp_hold_start_reg[] = {  
    /*寄存器地址, 寄存器值*/  
};  
static struct sensor_i2c_reg_tab xxx_grp_hold_start_tab = {  
    .settings = xxx_grp_hold_start_reg,  
    .size = ARRAY_SIZE(xxx_grp_hold_start_reg),  
};  
static SENSOR_REG_T xxx_grp_hold_end_reg[] = {  
    /*寄存器地址, 寄存器值*/  
};  
static struct sensor_i2c_reg_tab xxx_grp_hold_end_tab = {  
    .settings = xxx_grp_hold_end_reg,  
    .size = ARRAY_SIZE(xxx_grp_hold_end_reg),  
};  
static struct sensor_aec_i2c_tag xxx_aec_info = {  
    .slave_addr = (I2C_SLAVE_ADDR >> 1),  
    .addr_bits_type = SENSOR_I2C_REG_16BIT,  
    .data_bits_type = SENSOR_I2C_VAL_8BIT,  
    .shutter = & sc1300cs_shutter_tab,  
    .again = & sc1300cs_again_tab,  
    .dgain = & sc1300cs_dgain_tab,  
    .frame_length = & sc1300cs_frame_length_tab,  
    .grp_hold_start = &xxx_grp_hold_start_tab,  
    .grp_hold_end = &xxx_grp_hold_end_tab,  
};
```

需要查看Sensor spec, 确定当前Sensor是否支持Group hold功能。

- 如果支持, 则填写相应的寄存器地址和值。
- 如果不支持, 不需要填写寄存器地址和值, 保持xxxxx_grp_hold_start_reg[]和xxxxx_grp_hold_end_reg[]结构体数组内为空即可。

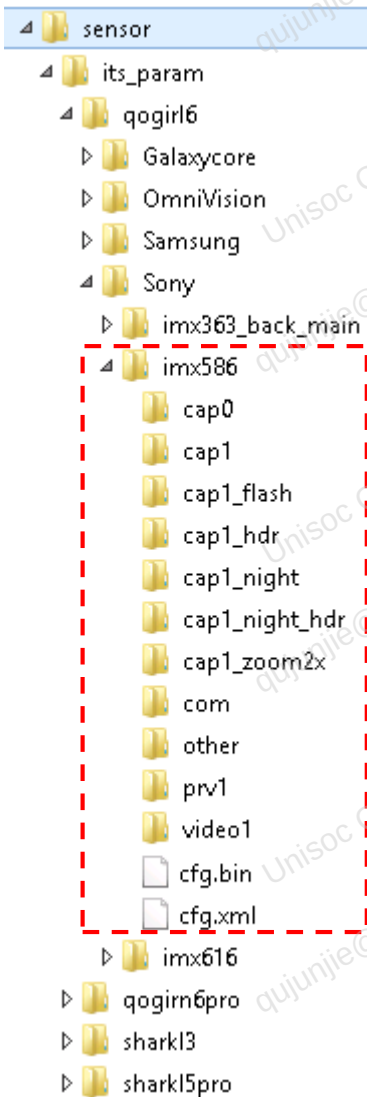
02

Sensor驱动配置

Tuning参数配置



路径为sprdroid13_main/vendor/sprd/modules/libcamera/sensor/its_param/, 以ums9230_4h10后主摄为例。



添加Tuning参数文件夹

- 需与sensor_config.xml中TuningName同名。
- 第一次点亮Sensor, 请使用Auto Tuning工具生成初始参数, 具体请参考《Camera Auto Tuning工具使用指导手册》。

将Tuning参数文件夹下所有文件复制到pac的vendor/firmware目录下 (将bin文件打包到pac)。

对应文件sprdroid13_main/vendor/sprd/modules/libcamera/libcam_device.mk内容如下:

```
ifndef $(filter $(TARGET_BOARD_PLATFORM), ums9230),
PRODUCT_COPY_FILES += $(call find-copy-subdir-files,*, vendor/sprd/modules/libcamera/sensor/its_param/qogirl6/Sony/imx586, vendor/firmware/imx586) \
$(call find-copy-subdir-files,*, vendor/sprd/modules/libcamera/sensor/its_param/qogirl6/Sony/imx616, vendor/firmware/imx616) \
$(call find-copy-subdir-files,*, vendor/sprd/modules/libcamera/sensor/its_param/qogirl6/Sony/imx363_back_main, vendor/firmware/imx363_back_main) \
$(call find-copy-subdir-files,*, vendor/sprd/modules/libcamera/sensor/its_param/qogirl6/Samsung/s5kgw1sp03, vendor/firmware/s5kgw1sp03) \
$(call find-copy-subdir-files,*, vendor/sprd/modules/libcamera/sensor/its_param/qogirl6/OmniVision/ov08a10, vendor/firmware/ov08a10) \
$(call find-copy-subdir-files,*, vendor/sprd/modules/libcamera/sensor/its_param/qogirl6/Samsung/s5kjnl, vendor/firmware/s5kjnl) \
$(call find-copy-subdir-files,*, vendor/sprd/modules/libcamera/sensor/its_param/qogirl6/OmniVision/ov8856_back, vendor/firmware/ov8856_back)
endif
```


02

Sensor驱动 配置 模组配置



模组属性字段

文件：sensor_config.xml

如sprdroid13_main/device/sprd/qogirl6/ums9230_4h10/module/camera/sensor_config.xml。

```
<CameraModuleCfg>
  <SlotId>0</SlotId>
  <SensorName>xxx</SensorName>
  <Facing>BACK</Facing>
  <Orientation>90</Orientation>
  <Resource_cost>55</Resource_cost>
  <SensorRole>dualcam_master</SensorRole>
  <VCM>
    <AfName>zzz</AfName>
    <Mode>0</Mode>
  </VCM>
  <OTP>
    <E2prom>
      <OtpName>general</OtpName>
      <I2cAddr>0xa0</I2cAddr>
      <E2promNum>1</E2promNum>
      <E2promSize>8192</E2promSize>
    </E2prom>
  </OTP>
  <TuningParameter>
    <TuningName>xxx</TuningName>
  </TuningParameter>
</CameraModuleCfg>
```

Sensor属性字段

- SensorName：与Sensor驱动文件夹名称相同
- Facing：Sensor朝向
- Orientation：Sensor安装角度
- Resource_cost：Sensor资源占有比例，0~100
- SensorRole：Sensor角色，如不配置默认为单摄

VCM属性字段

- AfName：与AF驱动文件夹名称相同
- Mode：AF工作模式，默认为0，暂未使用

OTP属性字段

- OtpName：与OTP驱动文件夹名称相同
- I2cAddr：E2prom I2C地址
- E2promNum：E2prom数量及存放方式
- E2promSize：E2prom大小，单位byte

Tuning参数属性字段

- TuningName：与its_param目录下Tuning参数文件夹名称相同

注意：

- 如没有AF或OTP，则不需配置对应字段。但Sensor属性字段必须配置。
- OtpName一般配置general，如使用OTP 2.0方案则配置general_xml。
- 如无法获取Tuning参数，可将TuningName设置为default，仅用于点亮，建议使用Auto Tuning工具生成的参数。

SensorRole



文件：sensor_config.xml

如sprdroid13_main/device/sprd/qogirl6/ums9230_4h10/module/camera/sensor_config.xml。

SensorRole参数配置见下表：

```
<SensorRole>dualcam_master</SensorRole>
<SensorRole>multicam_wide</SensorRole>
```

类型	配置项	说明
单摄	single_ir	● 不配置SensorRole，默认为单摄。 ● 同一Sensor最多可配置8个SensorRole。 ● 同一Facing配置下，不同Sensor间，同一SensorRole只能配置一个。如果配置了多个，仅第一个有效。但可以在其他Facing下配置该SensorRole。 ● 对于同一类型的一组SensorRole，例如 multicam_superwide/multicam_wide/multicam_tele，同一Sensor只能配置一个。如果配置了多个，最后一个有效。但可以为其他Sensor配置该组SensorRole。 ● SensorRole配置必须放置在Resource_cost之后。 ● SensorRole配置无顺序要求。
	single_macro	
双摄	dualcam_master	
	dualcam_slave	
三摄	multicam_superwide	
	multicam_wide	
	multicam_tele	
3D结构光	stl3d_rgb	
	stl3d_ir_left	
	stl3d_ir_right	

03

OTP驱动配置



路径为sprdroid13_main/vendor/sprd/modules/libcamera/sensor/otp_drv/driver.

- 展锐OTP方案：按照展锐平台OTP烧录规范，数据存储在E2prom中，建议使用general驱动。
- 个性化OTP方案：需另行编写OTP驱动xxx_yyy，并将OtpName配置为xxx_yyy。

下图为展锐OTP方案后双摄模组主摄像头的OTP配置示例，此时该主摄与辅摄共用EEPROM。

```
<CameraModuleCfg>
<SlotId>0</SlotId>
<SensorName>xxx</SensorName>
<Facing>BACK</Facing>
<Orientation>90</Orientation>
<Resource_cost>55</Resource_cost>
<SensorRole>dualcam_master</SensorRole>
<VCM>
  <AfName>zzz</AfName>
  <Mode>0</Mode>
</VCM>
<OTP>
  <E2prom>
    <OtpName>general</OtpName>
    <I2cAddr>0xa0</I2cAddr>
    <E2promNum>1</E2promNum>
    <E2promSize>8192</E2promSize>
  </E2prom>
</OTP>
<TuningParameter>
  <TuningName>xxx</TuningName>
</TuningParameter>
</CameraModuleCfg>
```

➤ 使用平台OTP 1.1，建议配置为3

➤ 使用个性化OTP，建议配置为0

```
enum eeprom_num_t {
  SINGLE_CAM_ONE_EEPROM = 0,
  DUAL_CAM_ONE_EEPROM = 1,
  DUAL_CAM_TWO_EEPROM = 2,
  MULTICAM_INDEPENDENT_EEPROM = 3,
};
```

0: 单摄，一个EEPROM
1: 双摄，共用一个EEPROM
2: 双摄，两个独立的EEPROM
3: 多摄，各自独立的EEPROM

libcamera/sensor/otp_drv/otp_info.h

个性化OTP 1/3

如不使用展锐OTP烧录方案，而是使用个性化OTP。例如，其他平台厂商的OTP烧录方案，或Sensor OTP。则展锐平台通用OTP驱动general不适用，需要单独编写OTP驱动代码。

以Sensor OTP为例，可使用以下两种方式应用Sensor OTP：

- 将Sensor OTP的处理放在Sensor driver中
- 借助平台OTP流程处理Sensor OTP

方式	优点	缺点
Sensor driver	不需要单独维护OTP驱动，代码量小	一般每次进Camera都会从寄存器中读取OTP，增加启动时间
平台OTP流程	开机扫描时从寄存器中读取OTP，存储到内存中。后续进入Camera，只需要从内存中读取OTP，非常迅速	需按照平台OTP架构嵌入Sensor OTP处理，代码量大

如使用其他平台厂商的OTP烧录方案，应用OTP方式与Sensor OTP类似，只是数据源不存储在Sensor寄存器中，而是存储在EEPROM中。

个性化OTP 2/3

- Sensor驱动中读取Sensor OTP

路径: sprdroid13_main/vendor/sprd/modules/libcamera/sensor/sensor_drv

文件: sensor_xxx_mipi_raw.c

函数: static cmr_int xxx_drv_access_val(cmr_handle handle, cmr_uint param)

```
static cmr_int xxx_drv_access_val(cmr_handle handle, cmr_uint param)
{
    cmr_int ret = SENSOR_FAIL;
    SENSOR_VAL_T *param_ptr = (SENSOR_VAL_T *)param;

    SENSOR_IC_CHECK_HANDLE(handle);
    SENSOR_IC_CHECK_PTR(param_ptr);
    struct sensor_ic_drv_cxt *sns_drv_cxt = (struct sensor_ic_drv_cxt *)handle;
    SENSOR_LOGI("sensor xxx: param_ptr->type=%x", param_ptr->type);

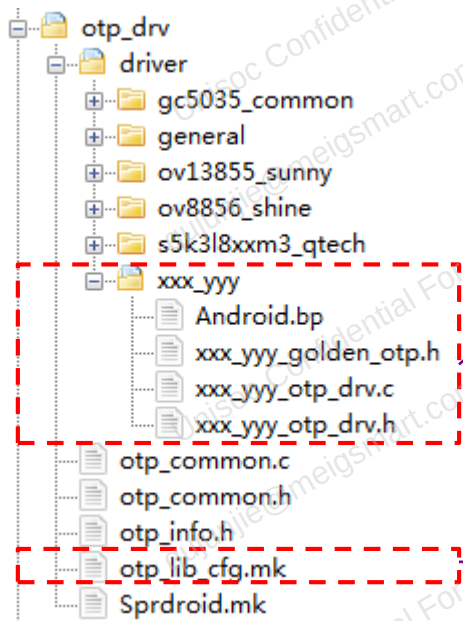
    switch(param_ptr->type)
    {
        case SENSOR_VAL_TYPE_GET_STATIC_INFO:
            ret = xxx_drv_get_static_info(handle, param_ptr->pval);
            break;
        .....
#ifdef FEATURE_OTP
        case SENSOR_VAL_TYPE_READ_OTP:
            ret = xxx_yyy_identify_otp(handle, param_ptr);
            break;
#endif
        default:
            break;
    }
}
```

在Sensor驱动的access value接口中添加case
SENSPR_VAL_TYPE_READ_OTP, 用于读取
Sensor OTP

个性化OTP 3/3

- 通过平台OTP流程读取Sensor OTP

路径: sprdroid13_main/vendor/sprd/modules/libcamera/sensor/otp_drv/driver



添加OTP驱动文件夹

- xxx: Sensor Name
- yyy: Module Vendor

xxx_yyy需与sensor_config.xml中OtpName同名, 驱动文件请参照工程自带驱动添加。

将so打包到pac

```
PRODUCT_PACKAGES += libotp_general \
                    libotp_s5k5e8yx_jd \
                    libotp_ov13855 \
                    libotp_gc5035_common \
                    libotp_s5k4h7_tsp \
                    libotp_xxx_yyy
```

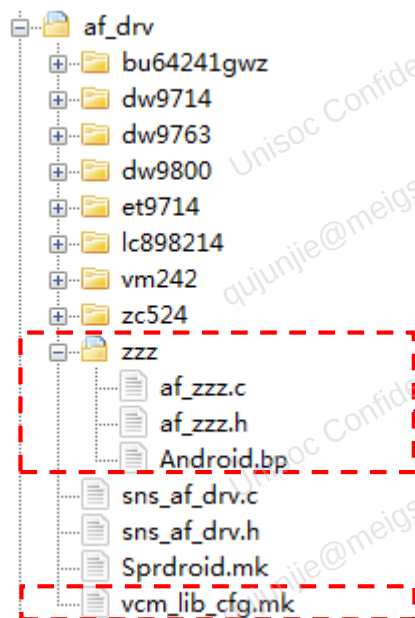
04

AF驱动配置



驱动文件

路径: sprdroid13_main/vendor/sprd/modules/libcamera/sensor/af_drv/



添加AF驱动文件夹

zzz: VCM Driver IC Name, 需与 sensor_config.xml中AfName同名, 驱动文件请参照工程自带驱动添加。

将so打包到pac

```
PRODUCT_PACKAGES += libvcm_dw9714p \  
libvcm_lc898213 \  
libvcm_dw9768v \  
libvcm_dw9800 \  
libvcm_dw9714 \  
libvcm_dw9718s \  
libvcm_zc524 \  
libvcm_zzz
```


05

Flash驱动配置

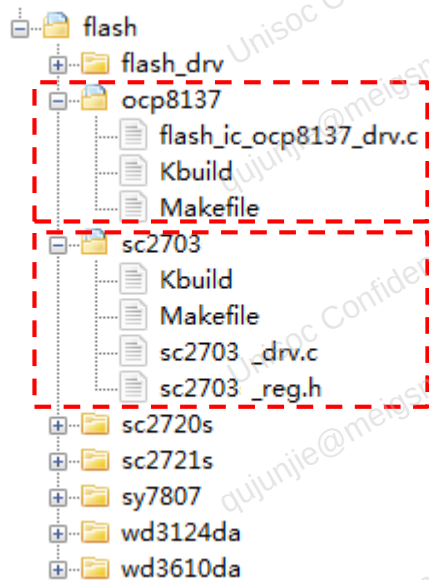


 紫光展锐
UNISOC

● 内置PMIC

- 外置闪光灯IC
 - ✓ GPIO控制, 例如AW3641
 - ✓ GPIO+I2C控制, 例如OCP8137, AW3648
 - ✓ PWM波控制, PWM模式, 使用脉冲控制Flash IC工作, 例如SY7807, WD3124DA
- 屏幕LCD做补光

路径为sprdroid13_main/bsp/modules/common/camera/flash/，以ocp8137和sc2703为例，添加驱动文件如下左图所示。对应DTS文件为sprdroid13_main/bsp/kernel5.4/kernel5.4/arch/arm64/boot/dts/sprd/ums512-1h10-overlay.dts，如下右图所示。



ocp8137

```
flash_ic: flash-ic@63 {
    compatible = "sprd,flash-ocp8137";
    reg = <0x63>;
    sprd,flash-ic = <8137>;
    sprd,torch = <1>;
    sprd,preflash = <1>;
    sprd,highlight = <1>;
    sprd,torch-level = <128>;
    sprd,preflash-level = <128>;
    sprd,highlight-level = <128>;
    sprd,lvfm-enable = <1>;
    flash-torch-en-gpios = <&ap_gpio 88 0>;
    flash-chip-en-gpios = <&ap_gpio 89 0>;
    flash-en-gpios = <&ap_gpio 76 0>;
    flash-sync-gpios = <&ap_gpio 141 0>;
};
```

sc2703

```
flash: sc2703-flash@4a {
    compatible = "sprd,sc2703-flash";
    reg = <0x4a>;
    flash-chip-en-gpios = <&ap_gpio 137 0>;
};
```

ko配置

以ocp8137和sc2703为例，ko路径为

sprdroid13_main/device/sprd/mpool/module/others/bsp/mfeature/kernel/kernel5.4/msoc/sharkl5Pro/ko/。

km.mk

```
RECOVERY_KERNEL_MODULES += \  
$(BSP_KERNEL_MODULES_OUT)/core.ko \  
$(BSP_KERNEL_MODULES_OUT)/gpio.ko \  
$(BSP_KERNEL_MODULES_OUT)/pinctrl.ko \  
$(BSP_KERNEL_MODULES_OUT)/pwm_bl.ko\  
$(BSP_KERNEL_MODULES_OUT)/sprd_flash_drv.ko \  
$(BSP_KERNEL_MODULES_OUT)/flash_ic_ocp8137.ko \  
$(BSP_KERNEL_MODULES_OUT)/flash_ic_sc2703.ko \  
$(BSP_KERNEL_MODULES_OUT)/sprd_camsys_pw_domain.ko \  
$(BSP_KERNEL_MODULES_OUT)/sprd-ion.ko \  
$(BSP_KERNEL_MODULES_OUT)/unisoc-iommu.ko \  

```

```
BOARD_VENDOR_KERNEL_MODULES += \  
$(BSP_KERNEL_MODULES_OUT)/core.ko \  
$(BSP_KERNEL_MODULES_OUT)/gpio.ko \  
$(BSP_KERNEL_MODULES_OUT)/pinctrl.ko \  
$(BSP_KERNEL_MODULES_OUT)/sfp_core.ko \  
$(BSP_KERNEL_MODULES_OUT)/microarray_fp.ko \  
$(BSP_KERNEL_MODULES_OUT)/cpufreq_userspace.ko \  
$(BSP_KERNEL_MODULES_OUT)/pwm_bl.ko\  
$(BSP_KERNEL_MODULES_OUT)/sprd_flash_drv.ko \  
$(BSP_KERNEL_MODULES_OUT)/flash_ic_ocp8137.ko \  
$(BSP_KERNEL_MODULES_OUT)/flash_ic_sc2703.ko \  

```

modules.load

```
pwm-sprd.ko  
sprd_flash_drv.ko  
flash_ic_ocp8137.ko  
flash_ic_sc2703.ko  
sprd_sensor.ko  
sprd_camera.ko
```

针对PWM方式控制Flash，需要确保pwm-sprd.ko的加载顺序在sprd_flash_drv.ko之前。

sprdroid13_main/bsp/device/sharkl5Pro/androidt/common/modules.cfg

```
elif [ $BSP_KERNEL_VERSION == "kernel5.4" ]; then  
BSP_MODULES_LIST="  
sample.ko  
synaptics_dsx_td4310.ko  
ssd20xx_ts.ko  
mali_gondul.ko  
sprdbt_tty.ko  
microarray_fp.ko  
sprd_fm.ko  
sprd-gsp.ko  
sprd-drm.ko  
sprd-ion.ko  
sprd_camsys_pw_domain.ko  
sprd_flash_drv.ko  
flash_ic_ocp8137.ko  
flash_ic_sc2703.ko  
sprd_sensor.ko  
sprd_camera.ko  
sprd_cpp.ko  
"
```

sprdroid13_main/vendor/sprd/modules/libcamera/kiface/init.sprd_flash.rc

```
on sprd_flash.ko  
insmod ${ro.vendor.ko.mount.point}/socko/flash_ic_sc2703.ko  
insmod ${ro.vendor.ko.mount.point}/socko/flash_ic_ocp8137.ko  
insmod ${ro.vendor.ko.mount.point}/socko/flash_ic_sy7807.ko  
insmod ${ro.vendor.ko.mount.point}/socko/flash_ic_sc2720.ko
```


Flash通用功能与流程在flash_drv.c中实现，路径为sprdroid13_main/bsp/modules/common/camera/flash/flash_drv。

```
struct sprd_flash_driver_ops {  
    int (*open_torch)(void *drv, uint8_t idx); // 开启手电筒模式  
    int (*close_torch)(void *drv, uint8_t idx); // 关闭手电筒模式  
    int (*open_preflash)(void *drv, uint8_t idx); // 开启预闪  
    int (*close_preflash)(void *drv, uint8_t idx); // 关闭预闪  
    int (*open_highlight)(void *drv, uint8_t idx); // 开启主闪  
    int (*close_highlight)(void *drv, uint8_t idx); // 关闭主闪  
    int (*cfg_value_torch)(void *drv, uint8_t idx, struct sprd_flash_element *element); // 设置手电筒模式亮度  
    int (*cfg_value_preflash)(void *drv, uint8_t idx, struct sprd_flash_element *element); // 设置预闪亮度  
    int (*cfg_value_highlight)(void *drv, uint8_t idx, struct sprd_flash_element *element); // 设置主闪亮度
```

Flash相关宏配置

Flash相关宏可以在下面两个文件中进行配置，且md.mk优先级高。

- sprdroid13_main/device/sprd/sharkl5Pro/ums512_1h10/module/camera/md.mk

✓ 双色温闪光灯配置

```
TARGET_BOARD_CAMERA_FLASH_LED_0 := true  
TARGET_BOARD_CAMERA_FLASH_LED_1 := true
```

设置为true或false

✓ 前摄闪光灯模式配置

```
TARGET_BOARD_FRONT_CAMERA_FLASH_TYPE := lcd
```

- sprdroid13_main/device/sprd/mpool/module/odm/camera/msoc/qogirl6/qogirl6.mk

✓ 双色温闪光灯配置

```
TARGET_BOARD_CAMERA_FLASH_LED_0 ?= true  
TARGET_BOARD_CAMERA_FLASH_LED_1 ?= true
```

✓ 前摄闪光灯模式配置

```
TARGET_BOARD_FRONT_CAMERA_FLASH_TYPE ?= lcd
```

- lcd: LCD补光
- led: LED补光
- flash: Flash模式

06 驱动编译和 下载 驱动编译



1. 进入工程根目录，设置环境变量：

source build/envsetup.sh

2. 选择根目录编译选项（图1为UMS9230平台示例）：

lunch <boardname>

3. 安装kernel header，在根目录执行：

kheader

4. 进入BSP目录，设置BSP环境变量：

source build/envsetup.sh

5. 选择BSP编译选项（图2为UMS9230平台示例）：

lunch <boardname_bsp>

6. 编译整个工程，返回根目录执行：

make

```
122. ums9230_4h10_Natv-userdebug-gms
123. ums9230_4h10_Natv-userdebug-native
124. ums9230_4h10_Skynet-userdebug-gms
125. ums9230_4h10_Skynet-userdebug-native
126. ums9230_4h10_cmcc-userdebug-gms
127. ums9230_4h10_cmcc-userdebug-native
128. ums9230_4h10_ctcc-userdebug-gms
129. ums9230_4h10_ctcc-userdebug-native
130. ums9230_4h10_go2g-userdebug-gms
131. ums9230_4h10_go2g-userdebug-native
132. ums9230_4h10_go2g_nosec-userdebug-native
133. ums9230_4h10_nosec-userdebug-native
134. ums9230_6h10_Natv-userdebug-gms
135. ums9230_6h10_Natv-userdebug-native
```

图1 lunch根目录

```
58. ums9230_1h10_Skynet-userdebug-androidt
59. ums9230_1h10_lwfq-userdebug-androidt
60. ums9230_1h10_nosec-userdebug-androidt
61. ums9230_4h10_Natv-userdebug-androidt
62. ums9230_4h10_cmcc-userdebug-androidt
63. ums9230_4h10_nosec-userdebug-androidt
64. ums9230_4h10_Skynet-userdebug-androidt
65. ums9230_4h10_ctcc-userdebug-androidt
66. ums9230_4h10_go2g-userdebug-androidt
67. ums9230_6h10_Natv-userdebug-androidt
```

图2 BSP根目录

单独编译Camera模块

1. 进入工程根目录，设置环境变量：

`source build/envsetup.sh`

2. 选择根目录编译选项（图1为UMS9230平台示例）：

`lunch <boardname>`

3. 安装kernel header，在根目录执行：

`kheader`

4. 进入vendor/sprd/modules/libcamera目录，执行编译指令：

`mm或mma`

```
122. ums9230_4h10_Natv-userdebug-gms
123. ums9230_4h10_Natv-userdebug-native
124. ums9230_4h10_Skynet-userdebug-gms
125. ums9230_4h10_Skynet-userdebug-native
126. ums9230_4h10_cmcc-userdebug-gms
127. ums9230_4h10_cmcc-userdebug-native
128. ums9230_4h10_ctcc-userdebug-gms
129. ums9230_4h10_ctcc-userdebug-native
130. ums9230_4h10_go2g-userdebug-gms
131. ums9230_4h10_go2g-userdebug-native
132. ums9230_4h10_go2g_nosec-userdebug-native
133. ums9230_4h10_nosec-userdebug-native
134. ums9230_6h10_Natv-userdebug-gms
135. ums9230_6h10_Natv-userdebug-native
```

图1 lunch根目录

单独编译Flash ko

1. 从工程根目录进入BSP目录，设置BSP环境变量：

source build/envsetup.sh

2. 选择BSP编译选项（图1为UMS9230平台示例）：

lunch <boardname_bsp>

3. 编译通用Flash驱动ko，在BSP下执行：

make modules -m sprd_flash_drv.ko

4. 编译具体Flash驱动ko，在BSP下执行：

make modules -m flash_ic_ocp8137.ko

make modules -m flash_ic_sc2703.ko

```
58. ums9230_1h10_Skynet-userdebug-androidt
59. ums9230_1h10_lwfq-userdebug-androidt
60. ums9230_1h10_nosec-userdebug-androidt
61. ums9230_4h10_Natv-userdebug-androidt
62. ums9230_4h10_cmcc-userdebug-androidt
63. ums9230_4h10_nosec-userdebug-androidt
64. ums9230_4h10_Skynet-userdebug-androidt
65. ums9230_4h10_ctcc-userdebug-androidt
66. ums9230_4h10_go2g-userdebug-androidt
67. ums9230_6h10_Natv-userdebug-androidt
```

图1 BSP根目录

1. 进入工程根目录，设置环境变量：

source build/envsetup.sh

2. 选择根目录编译选项（图1为UMS9230平台示例）：

lunch <boardname>

3. 编译boot.img，在根目录执行：

make bootimage

4. 编译dtbo.img，在根目录执行：

make dtboimage

```
122. ums9230_4h10_Natv-userdebug-gms
123. ums9230_4h10_Natv-userdebug-native
124. ums9230_4h10_Skynet-userdebug-gms
125. ums9230_4h10_Skynet-userdebug-native
126. ums9230_4h10_cmcc-userdebug-gms
127. ums9230_4h10_cmcc-userdebug-native
128. ums9230_4h10_ctcc-userdebug-gms
129. ums9230_4h10_ctcc-userdebug-native
130. ums9230_4h10_go2g-userdebug-gms
131. ums9230_4h10_go2g-userdebug-native
132. ums9230_4h10_go2g_nosec-userdebug-native
133. ums9230_4h10_nosec-userdebug-native
134. ums9230_6h10_Natv-userdebug-gms
135. ums9230_6h10_Natv-userdebug-native
```

图1 lunch根目录

06 驱动编译和 下载 驱动下载



下载Sensor/AF/OTP驱动



以ums9230_4h10为例，新增或修改Sensor/AF/OTP驱动后，编译生成的库文件输出目录：
sprdroid13_main/out/target/product/ums9230_4h10/odm/lib/。

新增或修改项	编译生成库文件	库文件在设备中存储位置	将库文件下载到设备
Sensor驱动xxx	libsensor_xxx.so	odm/lib	adb root adb remount adb push ***.so odm/lib adb reboot (***.so指对应的库文件名称, 存放在电脑本地)
AF驱动zzz	libvcm_zzz.so		
OTP驱动xxx_yyy	libotp_xxx_yyy.so		

下载sensor_config.xml



sensor_config.xml为免编译配置，新增或更换Sensor/AF/OTP驱动，修改驱动名称及其他模组属性，需要更新sensor_config.xml。

以ums9230_4h10为例，sensor_config.xml位置：

- 代码中：sprdroid13_main/device/sprd/qogirl6/ums9230_4h10/module/camera/sensor_config.xml
- 设备中：odm/etc/sensor_config.xml

将sensor_config.xml下载到设备（sensor_config.xml存放在电脑本地）。

adb root

adb remount

adb push sensor_config.xml odm/etc

adb reboot

将sensor_config.xml文件复制到pac的odm/etc/目录下（将xml文件打包到pac）

```
#camera sensor config
PRODUCT_COPY_FILES += \
    $(call md-board-overlay,camera/sensor_config.xml):$(TARGET_COPY_OUT_ODM)/etc/sensor_config.xml
```

sprdroid13_main/device/sprd/mpool/module/odm/camera/msoc/qogirl6/qogirl6.mk

或sprdroid13_main/device/sprd/sharkl5Pro/ums512_1h10/module/camera/md.mk

- mpool为芯片级（例如UMS512和UMS9230）的通用编译配置，具体board级（例如ums512_1h10和ums9230_4h10）的配置比mpool优先级高，会覆盖mpool下的配置。
- sensor_config.xml打包命令在芯片平台上一概已经集成好了，不需要再添加。

下载Sensor Tuning参数

Android 13 Tuning参数已实现免编译，以bin和xml文件形式存在，如下图所示。

- Tuning参数在代码中位置：sprdroid13_main/vendor/sprd/modules/libcamera/sensor/its_param/
- Tuning参数在设备中位置：vendor/firmware/

以ums9230_4h10后主摄imx586为例，将Tuning参数下载到设备。

下载前请确认：

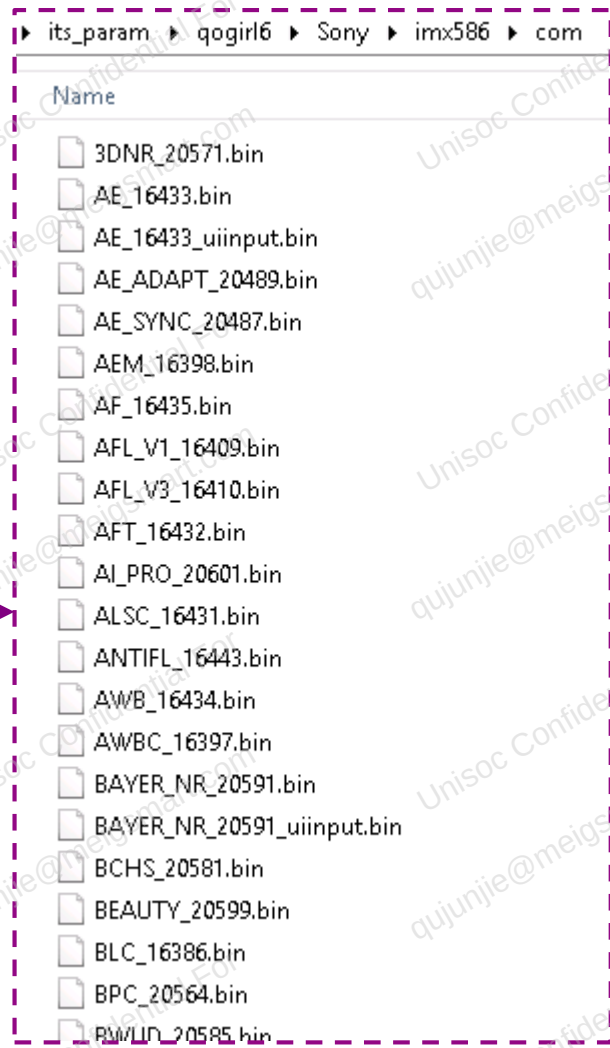
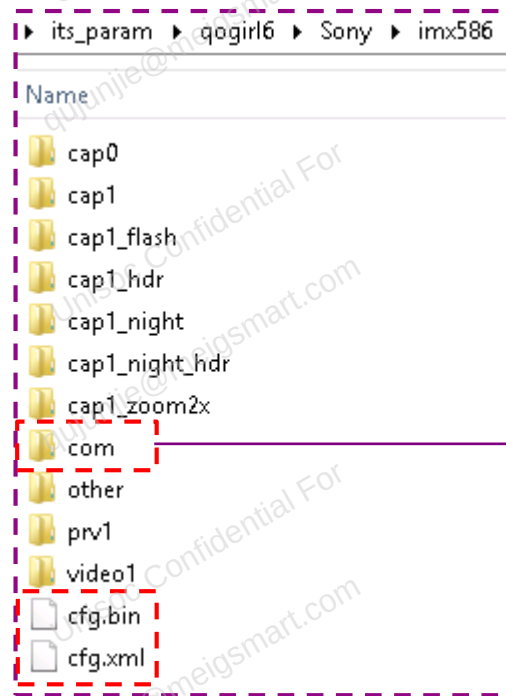
- Tuning参数存放在电脑本地imx586文件夹中
- 设备vendor/firmware/目录下已创建好imx586文件夹

adb root

adb remount

adb push .\imx586 vendor/firmware/imx586

adb reboot



也可使用IspTool工具下载参数，具体请参考IspTool工具包中的使用指导手册。

下载Flash驱动



以ums512_1h10为例，新增或修改Flash驱动后，编译生成的库文件输出目录：
sprdroid13_main/bsp/out/androidt/ums512_1h10_Natv/dist/modules/archive/。

新增或修改项	编译生成库文件	库文件在设备中存储位置	将库文件下载到设备
通用Flash驱动	sprd_flash_drv.ko	vendor_dlkm/lib/modules	adb root adb remount adb push ***.ko vendor_dlkm/lib/modules adb reboot (***.ko指对应的库文件名称，存放在电脑本地)
具体Flash驱动	flash_ic_ocp8137.ko		
	flash_ic_sc2703.ko		

下载DTS

以ums9230_4h10为例，修改DTS后，编译生成的image文件输出目录：

sprdroid13_main/out/target/product/ums9230_4h10/。

将DTS下载到设备：boot.img和dtbo.img存放在电脑本地。

1. 重启设备进入fastboot模式

adb reboot bootloader

2. 将boot.img下载到设备

fastboot flash boot boot.img

3. 将dtbo.img下载到设备

fastboot flash dtbo dtbo.img

4. 重启设备进入正常模式

fastboot reboot

也可使用展锐pac下载工具ResearchDownload下载image，具体请参考ResearchDownload工具包中的使用指导手册。

谢谢



本文件所含数据和信息都属于紫光展锐（上海）科技有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负责任与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

紫光展锐 UNISOC、、展讯、Spreadtrum、SPRD、锐迪科、RDA及其他紫光展锐的商标均为紫光展锐（上海）科技有限公司及/或其子公司、关联公司所有。

Bluetooth®文字商标和徽标为蓝牙技术联盟的注册商标，紫光展锐（上海）科技有限公司对此类商标的任何使用均已获得许可。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

本文档可能包含第三方内容，包括但不限于第三方信息、软件、组件、数据等。紫光展锐不控制且不对第三方内容承担任何责任，包括但不限于准确性、兼容性、可靠性、可用性、合法性、适当性、性能、不侵权、更新状态等，除非本文档另有明确说明。在本文档中提及或引用任何第三方内容不代表紫光展锐对第三方内容的认可、承诺或保证。

用户有义务结合自身情况，检查上述第三方内容的可用性。若需要第三方许可，应通过合法途径获取第三方许可，除非本文档另有明确说明。